

Heart Disease Prediction



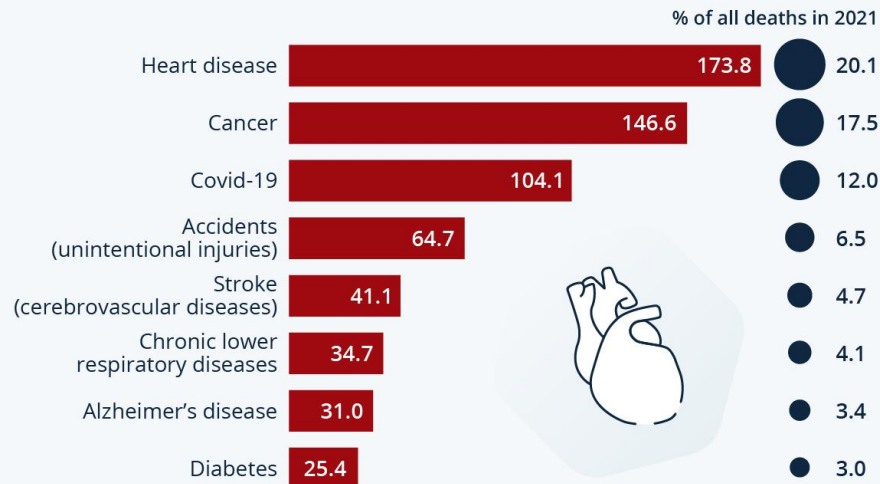
Christopher Haddad & Alejandro Olmedo
CS-UY 4563: Machine Learning

Why Heart Disease?

- Leading cause of death in the world (17.9 million deaths/year)
- Real-world relevance
 - Early prediction can save lives

One in Five Deaths in the U.S. Caused by Heart Disease

Number of people who died from the following causes in the United States per 100,000 of population*



* Mar. 2020-Oct. 2021 (latest final data). Age-adjusted death rate

Sources: CDC, NCHS

Dataset Overview

Heart Disease Prediction Dataset

2844

<> Code

Data Card Code (1340) Discussion (28) Suggestions (1)

heart.csv (35.92 kB)



Detail Compact Column

10 of 12 columns

About this file

The data contains 918 observations with 12 attributes.

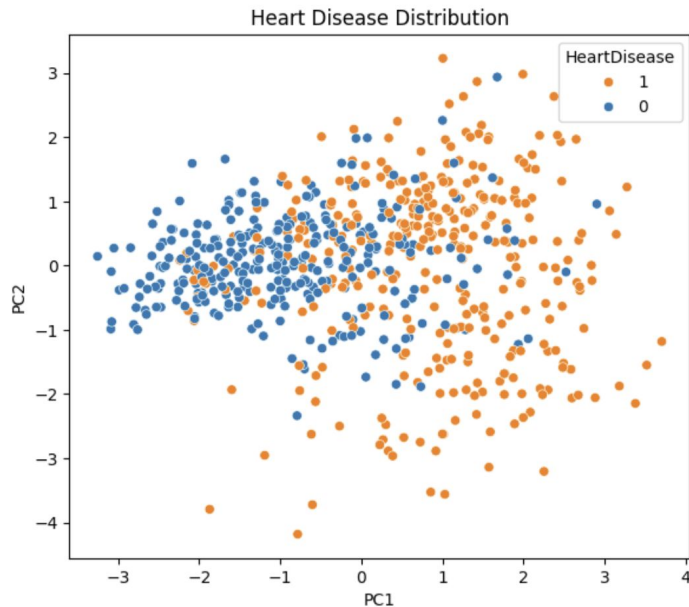
# Age	# Sex	# ChestPainType	# RestingBP	# Cholesterol	# FastingBS
age	sex	chest pain type	resting blood pressure	serum cholesterol	fasting blood sugar
	M 79% F 21%	ASY 54% NAP 22% Other (219) 24%			
28			0	0	0
48	M	ATA	148	289	0
49	F	NAP	168	188	0
37	M	ATA	138	283	0
48	F	ASY	138	214	0
54	M	NAP	158	195	0

Heart Disease Kaggle Dataset

- 918 Patients
- 11 Clinical Features
 - **Numerical:** Age, Resting BP, Cholesterol, Fasting BS, Max HR, Oldpeak
 - **Categorical:** Sex, Chest Pain Type, Resting ECG, Exercise Angina, ST Slope
- **Target variable:** Heart disease (1) or No Heart disease (0)

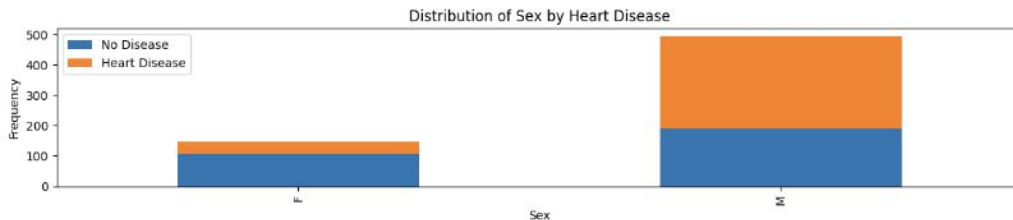
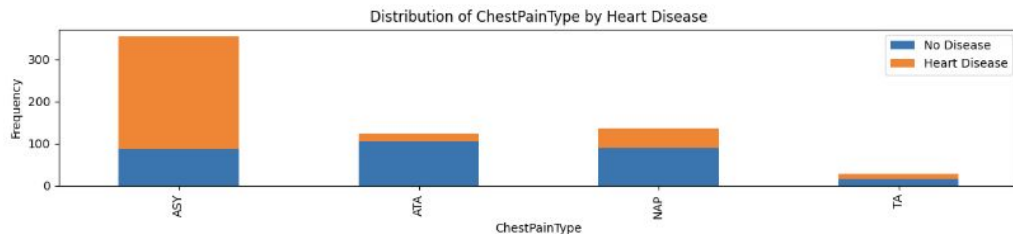
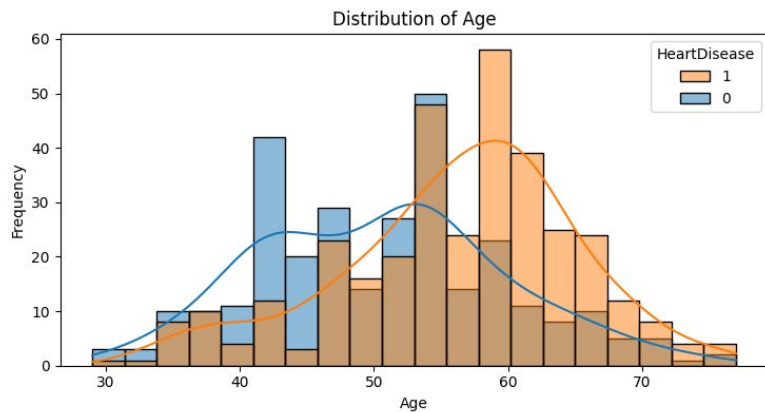
Preprocessing

- Ensured dataset contained no missing values
- Standardize using *StandardScaler* (0 mean, 1 standard deviation)
- One Hot encoded categorical features using *OneHotEncoder* for numerical representation
- Dataset split:
 - 70% Training
 - 15% Validation
 - 15% Test

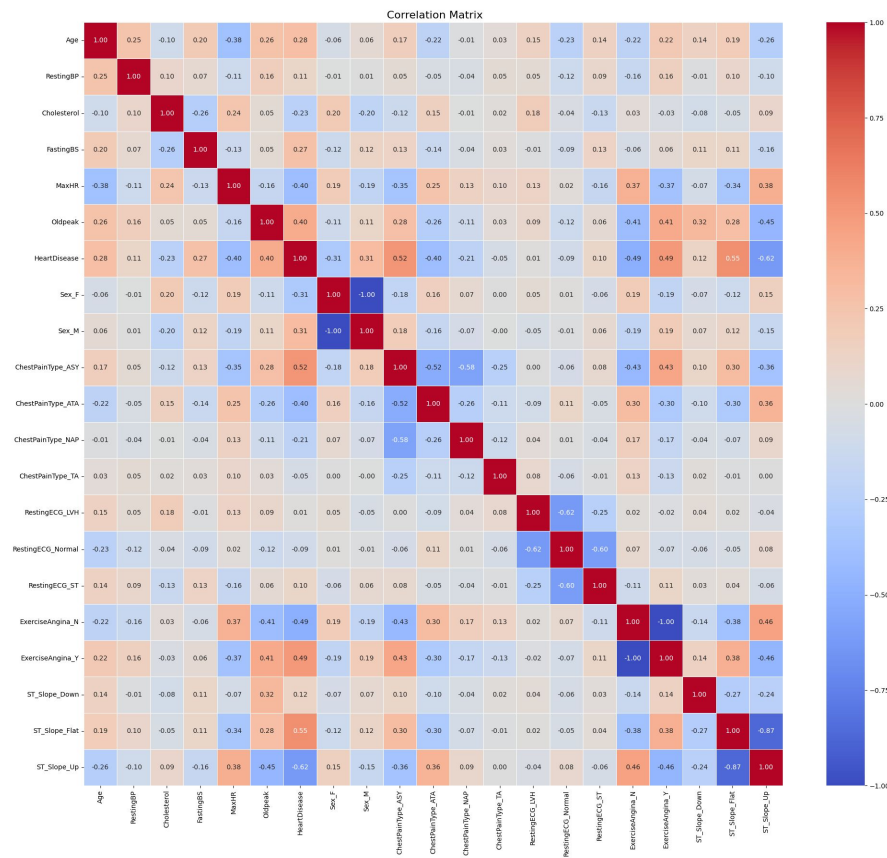


Unsupervised Learning

- Histograms of Numerical Features
- Bar Charts for Categorical Features



Correlation Matrix



Logistic Regression

Logistic Regression: Overview

- Transformations
 - Polynomial degree 2
 - Polynomial degree 3
 - Principal Component Analysis (PCA)
- Regularization (C)
 - Values: 0.001, 0.01, 0.1, 1.0, 10.0, and 100.0

Logistic Regression: Results

Top 5 Logistic Regression Models

Transformation	Regularization (C)	Test Accuracy	Test F-1 Score
No Transform	1.0	0.891	0.908
No Transform	10.0	0.891	0.908
No Transform	100.0	0.891	0.908
Polynomial Degree 2	0.1	0.891	0.91
Polynomial Degree 3	0.01	0.884	0.901

Support Vector Machine (SVM)

SVM: Overview

- Transformations
 - Polynomial degree 2
 - Principal Component Analysis (PCA)
 - Radial Basis Function Kernel (RBF)
- Regularization (C)
 - Values: 0.001, 0.01, 0.1, 1.0, 10.0, and 100.0

SVM Results

Top 5 SVM Models

Transformation	Regularization (C)	Test Accuracy	Test F-1 Score
Linear Kernel	1.0	0.891	0.908
Linear Kernel	10.0	0.891	0.908
Linear Kernel	100.0	0.891	0.908
Polynomial Degree 2	1.0	0.884	0.904
Linear Kernel	0.1	0.877	0.897

Neural Network

Neural Networks: Overview

- Transformations
 - One hidden layer (16 Neurons)
 - Two hidden layers (32-16 Neurons)
 - Three hidden layers (64-32-16 Neurons)
 - Radial Basis Function Kernel (RBF)
- Regularization (C)
 - Values: 0.001, 0.01, 0.1, 1.0, 10.0, and 100.0

Neural Networks: Results

Top 5 Neural Network Models

Transformation	Regularization (λ)	Test Accuracy	Test F-1 Score
NN_64-32-16	0.0000	0.8913	0.9080
NN_16	0.0001	0.8841	0.9024
NN_32-16	0.0010	0.8768	0.8944
NN_16	0.0010	0.8768	0.8944
NN_32-16	0.0001	0.8768	0.8944

Top 5 Models

Top 5 Overall Models

Model	Transformation	Regularization (C)	Test Accuracy	Test F-1 Score
Neural Network	NN_64-32-16	0.0000	0.8913	0.9080
SVM	Linear Kernel	100.0000	0.8913	0.9080
Logistic Regression	No Transform	1.0000	0.8913	0.9080
Logistic Regression	No Transform	10.0000	0.8913	0.9080
Logistic Regression	No Transform	100.0000	0.8913	0.9080

- Simpler models often generalized better
- Extra complexity (deep NNs, transformations) didn't always help
- Linear SVM and logistic regression were best overall

Future Improvements

- Try alternative transformations
 - Logistic Regression: Logit
 - Sigmoid kernel for SVM
- Experiment with the number of layers and neurons in Neural Network
- Split the dataset differently
 - Example: 80% Training, 10% Validation, 10% Test

Thank You For Listening!